Test Logs And Defects

This file collates the full content of the main log, delegated logs, and confirmed defect files for PDF generation.

issue-228-test-log.md

2026-06-24T17:43:48.8977160+01:00

- Establish the session folder, preserve the exact user goal in-repo, and prepare append-only artifacts before any substantive testing so the deployed-only review is auditable from the first action.

Created `docs/testing/20260624/issue-228-001/` with `defects/` and `screenshots/`, saved the full goal prompt to `issue-228-session-goal-prompt.md`, reviewed the issue/PR references, and noted the request mismatch between the text `PR #243` and the pasted PR link `#231`.

Issue `#228` and PR `#243` align in the actual repository because PR `#243` explicitly closes issue `#228`; PR `#231` is a separate earlier command-help PR that may still provide historical context for docs/help changes but is not the primary review target for this session.

2026-06-24T17:50:35.0000000+01:00

- Prove browser access and real interaction against the deployed test environment before the planning and delegation stages, because the session rules require browser interaction evidence prior to substantive testing.

Opened `https://eviltester.github.io/grid-table-editor/`, confirmed the landing page build metadata for branch `codex/228-improve-command-definition` and commit `8382b9e1947b`, then navigated to `https://eviltester.github.io/grid-table-editor/generat` In the deployed generator I entered `ProofValue` as the column name, entered regex `[A-Z]{4}`, tabbed out to commit the row state, clicked `Preview`, and captured screenshots at `screenshots/browser-proof-generator-initial.png` and `screenshots/browser-proof-generator-preview.png`.

Browser control is proven for this session. The deployed generator accepted the row edits and produced preview output with generated values such as `NVLO`, `YMJO`, and `LFPF`, which confirms that the application can be opened, edited, and exercised in-browser through MCP tooling rather than only being loaded passively.

2026-06-24T18:05:00.0000000+01:00

- Start Loop 1 broad coverage by sampling the primary enum and shared command-definition seams first, because PR #243 changes enum normalization while the deployed branch also contains broader command/help-definition updates that the user explicitly wants covered.

On generator.html I switched the schema row to domain and confirmed that the domain command picker list includes datatype.enum. I then switched to text mode and executed these schemas through Preview:

Status + enum(active,inactive,pending)

Method + awd.datatype.enum(GET,POST,PUT,PATCH)

I also navigated to the deployed datatype docs page at https://eviltester.github.io/grid-table-editor/s captured screenshots/main-loop1-datatype-doc.png, and inspected the accessibility snapshot and console output for docs/runtime consistency clues.

Initial Loop 1 observations:

- Positive: text-mode enum(...) generated only the authored enum members (active, inactive, pending).
- Positive: text-mode awd.datatype.enum(...) generated only the authored HTTP-method-like values (GET, POST, PUT, PATCH), which is a strong early sign that canonical domain enum normalization works in the deployed runtime.
- Positive: the generator's domain command picker visibly lists datatype.enum, so the new command is exposed in at least one UI command-selection surface.
- Suspicious: the deployed datatype docs page snapshot visibly exposed datatype.boolean content but did not expose a visible datatype.enum method section in the captured accessible content, despite the runtime and domain picker surfacing datatype.enum.
- Suspicious: the datatype docs page logged a 404 console resource failure and several structural issues including interactive elements inside summary, missing form labels, and Quirks Mode warnings; these may be pre-existing but deserve follow-up because this session explicitly covers docs/help/content quality.
- Gap identified: selecting datatype.enum directly through the domain picker combobox was less automatable than text-mode authoring in this first pass, so a follow-up pass should confirm the full row-mode picker-to-params workflow rather than relying only on text mode.

2026-06-24T18:20:00.0000000+01:00

- Run Loop 2 as an explicit idea-generation pass focused on enum syntax variants, docs/help drift, and cheap cross-surface checks that directly target

the normalization and shared-metadata risks in PR #243.

Loop 2 ideas generated and classified:

1. Execute now: test `datatype.enum(values="GET,POST,PUT")` in text mode.
2. Execute now: switch the named-values enum case back to row mode and inspect whether it normalizes into a plain enum row.
3. Execute now: sample a non-CSV export surface with enum data using `MARKDOWN`.
4. Execute now: verify by fetched HTML whether the deployed datatype docs actually contain `datatype.enum`.
5. Execute now: verify by fetched HTML whether the deployed image docs still mention removed command `urlLoremFlickr`.
6. Execute now: verify by fetched HTML whether deployed internet docs expose `internet.httpMethod` plus documented params such as `commonOnly` and `excludes`.
7. Execute now: verify whether the deployed method-picker UI spec docs mention `datatype.enum`.
8. Execute now: do a quick app-shell sanity pass and capture any console/runtime hints that could indicate help regression.
9. Defer: direct row-mode domain-picker selection workflow for `datatype.enum` because the UI automation path is more brittle and the enum cross-surface subagent owns the deeper row-mode workflow.
10. Defer: HTML/Gherkin export-specific enum checks because the enum cross-surface lane owns broader export-oriented sampling.

Actions taken:

- In generator text mode, entered `Method` + `datatype.enum(values="GET,POST,PUT")`.
- Switched output format to `MARKDOWN` and previewed the generated data.
- Switched back to row mode to inspect how the text-mode authored domain enum renders in the schema editor.
- Fetched deployed docs HTML for:
  - `site/docs/test-data/domain/datatype/`
  - `site/docs/test-data/domain/image/`
  - `site/docs/test-data/domain/internet/`
  - `site/docs/test-data/method-picker-ui-spec`
- Revisited `app.html` and checked the live console output.

Loop 2 observations and results:

- Positive: `datatype.enum(values="GET,POST,PUT")` generated only `GET`, `POST`, and `PUT` in Markdown preview.
- Positive: switching that schema back to row mode normalized it to `sourceType=enum` with visible row value `GET,POST,PUT`, which is strong evidence that named-values domain syntax round-trips into the intended public enum editing surface.

- Confirmed docs drift: fetched deployed datatype docs HTML contains `datatype.boolean` but does not contain `datatype.enum`.
- Confirmed removal success: fetched deployed image docs HTML does not contain `urlLoremFlickr`, while it does contain `urlPicsumPhotos`.
- Confirmed broader docs surface health: fetched deployed internet docs HTML contains `internet.httpMethod` and both `commonOnly` and `excludes`.
- Confirmed another docs/help gap: fetched deployed method-picker UI spec HTML does not mention `datatype.enum`.
- Suspicious app-shell signal: `app.html` logs `TODO: Create help for instructions-summary-title` in the live browser console, which suggests at least one help string path is still unfinished or debug-oriented in the deployed app surface.

2026-06-24T18:32:00.0000000+01:00

- Run Loop 3 as a normalization-and-error-handling pass, because the strongest remaining uncertainty was whether explicit enum parsing fails safely or silently.

Loop 3 ideas generated and classified:

1. Execute now: round-trip a valid named-values enum schema from text mode back to row mode, then back to text mode.
2. Execute now: confirm the public text representation after round-trip stays `enum(...)` rather than leaking `datatype.enum(...)`.
3. Execute now: probe malformed explicit enum text with `datatype.enum(unclosed`.
4. Execute now: confirm whether malformed explicit enum text produces a validation error or silently generates output.
5. Execute now: capture the normalized row-mode state after valid named-values enum syntax.
6. Execute now: capture the text-mode round-trip state after normalization.
7. Defer: shorthand `enum value1,value2` syntax retest because the enum subagent already confirmed one failing shorthand case.
8. Defer: direct app-side enum authoring parity because the enum subagent only partially covered the app shell and the main lane already has stronger generator evidence.
9. Defer: per-format HTML/Gherkin export checks because core runtime defects already emerged and export-specific verification adds less marginal value than the remaining final review work.
10. Defer: save/load schema file persistence for enum syntax because this session stayed focused on deployed runtime behavior and no file round-trip regression evidence appeared in earlier loops.

Actions taken:

- Starting from the working `datatype.enum(values="GET,POST,PUT")` schema, switched back to row mode and confirmed the row became `enum` with visible value `GET,POST,PUT`.
- Switched back to text mode and confirmed the public schema text rendered as `Method + enum(GET,POST,PUT)`.
- Replaced the schema text with malformed explicit enum input:

`Method datatype.enum(unclosed`

- Clicked `Preview` and inspected both preview text and table output.

Loop 3 observations and results:

- Positive: valid named-values domain syntax round-trips through the public editing surface exactly as intended, ending as public `enum(GET,POST,PUT)` text rather than leaking internal domain-command syntax.
- Positive: the normalized row-mode state for the valid case was clearly `sourceType=enum` with plain comma-separated values, which supports the PR's normalization goal.
- Confirmed defect: malformed explicit enum text does not raise a validation error. Instead, preview succeeds and emits repeated literal value `datatype.enum(unclosed`, which hides the parsing failure and silently produces bad data.
- This malformed-explicit-enum behavior is more severe than a wording issue because it converts a broken command-like input into apparently successful generation output.

2026-06-24T18:40:00.0000000+01:00

- Perform the mandatory final review loop over the story, PR scope, logs, subagent findings, command-family sampling, docs reviewed, examples tried, defects found, and remaining gaps so the session stops for a documented reason rather than by exhaustion.

Final review ideas generated and classified:

1. Execute now: re-verify that published datatype docs are missing `datatype.enum` by using the fetched HTML evidence already collected.
2. Execute now: re-verify that the named-values enum case round-trips to public `enum(...)` text.
3. Execute now: re-verify that malformed explicit enum text silently generates literal output.
4. Execute now: fold the completed subagent evidence into the final defect set and gap model.
5. Defer: live `datatype.enum(...)` selection through the row-mode domain picker because automation friction remained high and the text-mode/runtime evidence is already strong.

6. Defer: app-side enum authoring parity because the app shell was only partially covered and the remaining risk is narrower than the already confirmed generator defects.
7. Defer: deeper screen-reader announcement auditing because the responsive lane already found keyboard/focus defects and this session stayed browser-MCP-only.
8. Defer: broad finance/location/number positive sampling beyond `date.between` because the session already demonstrated representative validator coverage and had stronger higher-value defects elsewhere.
9. Defer: full export-surface parity across HTML/Gherkin/Code because core parsing/docs/accessibility issues already block recommendation.
10. Defer: load/save schema file persistence because this deployed-only session prioritized live runtime authoring, docs, help, and accessibility behavior.

Final review observations:

- Coverage is now broad enough for the actual PR/runtime risk: enum normalization, malformed explicit enum handling, docs/help drift, faker helper workflow regression, structured param validation, app-shell help signals, and responsive/accessibility behavior were all sampled with live evidence.
- Multiple loops were completed:
  - Loop 1 baseline and broad coverage
  - Loop 2 targeted normalization/docs checks
  - Loop 3 round-trip and malformed-enum checks
  - final review loop
- Recent loops continued producing real information through Loop 3, but the newest issues now cluster around already-identified themes instead of opening brand-new risk areas.
- Stopping is justified because the remaining open ideas are narrower follow-up coverage rather than blockers to understanding the current acceptability of the change set.

---

command-coverage-test-log.md

Command Coverage Test Log

2026-06-24T17:52:00.0000000+01:00

- Charter: sample representative positive command usage across broad command families on the deployed branch, with special attention to examples exposed by the UI and docs.

Techniques and heuristics to use: exploratory testing, risk-based sampling, equivalence partitioning, consistency checking across help/docs/runtime, and pairwise breadth across command families.

Expected focus: domain command families, faker/helper commands, newly added or newly surfaced commands, removed/deprecated commands, commands with validators, commands with structured params, and commands whose help/docs contain multiple examples.

2026-06-24T18:28:00.0000000+01:00

- Cover broad representative command families on the deployed branch using the strongest current runtime and docs surfaces, because the original command-coverage lane did not finish in time and the session still needs explicit command-family sampling evidence.

Actions taken:

- Executed regex in generator row mode with column `ProofValue` and pattern `[A-Z]{4}`, then previewed generated output.
- Executed enum/domain-normalization paths in generator text mode:
  - `enum(active,inactive,pending)`
  - `awd.datatype.enum(GET,POST,PUT,PATCH)`
  - `datatype.enum(values="GET,POST,PUT")`
- Switched the named-values enum case back to row mode and confirmed it normalized to `enum` with value `GET,POST,PUT`.
- Verified the generator domain picker visibly contains `datatype.enum`.
- Verified published docs coverage by fetched HTML for:
  - datatype docs
  - image docs
  - internet docs
  - method-picker UI spec docs
- Incorporated the UX lane's live faker-helper evidence for `helpers.rangeToNumber`.
- Incorporated the negative-validation lane's live structured-domain evidence for `date.between`.

Observations:

- Sampled command families/source types:
  - `regex`: positive runtime generation confirmed
  - `enum`: positive runtime generation confirmed in row mode and text mode
  - `domain`: `datatype.enum` is exposed in the picker and executes through text-mode domain syntax
  - `faker/helper`: `helpers.rangeToNumber` is discoverable but its documented object-shaped params are blocked by the params editor

  - structured-param domain validation: `date.between` rejects missing params, wrong primitive shape, and reversed bounds with visible blocking feedback
  - removed/deprecated docs check: image docs omit `urlLoremFlickr`
  - multi-example/structured docs check: internet docs include `internet.httpMethod` with `commonOnly` and `excludes`
- Strongest command-coverage defect signal from this pass:
  - malformed explicit enum syntax can be silently treated as literal generated output
  - `helpers.rangeToNumber` object params are taught in help but blocked in the UI
  - `datatype.enum` is present in runtime but absent from key published docs surfaces
- Deferred families/why:
  - broader faker-helper family sweep: deferred for time after the `helpers.rangeToNumber` contradiction was confirmed
  - app-side positive command execution across multiple source types: deferred because the app shell is denser and the generator gave faster issue-focused evidence
  - HTML/Gherkin export parity for enum: deferred to final-gap/risk notes after stronger core defects were already found

---

2026-06-24T17:56:22.1679242+01:00

- intent focused broad-sampling pass across deployed runtime, nested docs, and published command/help surfaces for issue #228 / PR #243 command coverage, aiming to hit domain, faker/helper, changed-docs, structured-param, and multi-example families quickly without local repo verification.
- actions used Playwright against https://eviltester.github.io/grid-table-editor/ and https://eviltester.github.io/grid-table-editor/generator.html to confirm the deployed test environment, inspect the generator runtime surfaces, and capture a screenshot of the live generator command surface. sampled the live generator UI source-type selector and related help affordances: enum, literal, regex, domain, faker; also sampled the runtime output-format surface and option/help controls to confirm broad published command exposure from the deployed app. fetched and reviewed live nested docs pages from the deployed site: test-data/domain/domain-test-data, test-data/faker-test-data, test-data/Schema-Definition, test-data/generate-to-file, and interfaces-and-deployment/cli-node-and-bun. checked for changed or stale terminology on sampled live docs pages, including migration guidance for helper usage and stale datatype.enum references.
- observations runtime surface looked broadly healthy in sampling: generator.html exposed the expected schema source families (enum, literal,

regex, domain, faker), a large published output-format matrix, and help/option controls. Screenshot saved at screenshots/command-coverage-generator-runtime.png. domain example coverage was strong on the live docs page: examples included person.firstName(), internet.email(), location.cardinalDirection(abbreviated=true), date.between(from=1577836800000, to=1659312000000), finance.iban(formatted=true, countryCode="GB"), and number.int(min=32, max=47). faker/helper coverage was also explicit on the live docs page: helpers.mustache(...), helpers.fake(...), and faker.location.cardinalDirection({ abbreviated: true }) were published, with page guidance saying helpers.* is faker-only and not part of the domain abstraction. changed-docs / migration coverage looked intentionally updated on the live domain docs: the page explicitly marks domain.helpers.fake("...") as a before-invalid example and redirects readers to faker.helpers.fake("...") or helpers.fake(...) in faker contexts. I did not find sampled live references to datatype.enum on the reviewed pages. structured-parameter coverage was strong on the live CLI page: --show-progress, --stream, --stream-threshold, --unsafe-faker-expressions, --trim-input, and --trim-input-fields were all documented alongside amend-specific parameters and behavior notes. multi-example surface coverage was strong on the live docs: the CLI page published multiple generate and amend examples, while Schema Definition published two-line, compact inline, and values-form examples such as Status: enum("Open","In Progress","Closed") and Browser: Chrome,Firefox,Safari. generate-to-file docs provided a second docs-example surface tying rule families together and explicitly listed Faker, RegEx, Literal, and Enum, including pairwise generation as part of the published generation workflow. deferred in this rapid pass: deeper per-domain drill-down pages beyond the top-level domain index, direct validation of every runtime help icon navigation target, and broader nested blog/help coverage. Those were deferred to keep this pass broad and fast.

---

negative-validation-test-log.md

Negative Validation Test Log

2026-06-24T17:52:00.0000000+01:00

- Charter: probe malformed parameter handling, validator behavior, and feedback quality across enum and non-enum command families in the deployed UI.

Techniques and heuristics to use: negative testing, boundary analysis, equivalence partitioning, row-mode versus text-mode comparison, and repeatability checks

for any suspect behavior.

Expected focus: malformed syntax, missing params, wrong-order bounds, enum-shape ambiguity, structured-parameter validation, and whether errors are visible, consistent, and actionable.

2026-06-24T17:51:00+01:00

- Intent: Probe structured domain validation for `date.between` in row mode, including missing required params, type misuse, and reversed bounds.
- Actions: Opened `generator.html`, switched the first schema row to `domain`, selected `date.between` via the method picker, set column name `createdAt`, opened the structured params editor, and exercised three negative cases: missing `from`, `from=abc,to=123`, and `from=1609459200000,to=1577836800000`.
- Observations: The deployed UI validates these cases immediately and specifically in row mode. Missing params show `Row 1: invalid domain params - Invalid keyword arguments: argument "from" is required`. Non-integer input shows `bare values are not allowed; wrap strings in quotes`, which is precise about the parser complaint but slightly indirect for an integer field because it does not explicitly say `expected integer`. Reversed bounds show `argument "from" must be less than or equal to argument "to"`. The structured editor keeps `Apply` disabled for the invalid cases, and the inline row status plus modal status stayed consistent in repeated checks.
- Screenshot: `screenshots/negative-validation-date-between-reversed-range.png`.

2026-06-24T17:53:30+01:00

- Intent: Sample text-mode validation and compare its feedback path against the row-mode `date.between` checks.
- Actions: Switched the schema editor to text mode and tried a single-line entry using `createdAt date.between(from=abc,to=123)`, then triggered Preview.
- Observations: This was an invalid text-mode setup because the editor placeholder expects alternating lines (`Column Name` then `rule`). The resulting message was generic rather than command-specific: `column createdAt date.between(from=abc,to=123) requires a data definition, use 'literal("")' for blank data`. I am not treating that as a product defect because the setup format was wrong, but it does show

that text-mode feedback is easier to misread when the user collapses the two-line schema shape into one line. A clean same-command text-mode comparison remains deferred.

---

docs-consistency-test-log.md

Docs Consistency Test Log

2026-06-24T17:52:00.0000000+01:00

- Charter: compare published docs, in-app help, method-picker content, and live runtime behavior for command families exposed on the deployed branch.

Techniques and heuristics to use: documentation testing, consistency/oracle checking, stale-content hunting, and representative example execution.

Expected focus: docs links, examples, removed commands, missing commands, stale wording, and mismatches between what docs/help claim and what the deployed runtime actually accepts or produces.

2026-06-24T17:48:30.0000000+01:00

- Intent: prove browser access to the deployed branch and capture the live publication metadata before checking docs/help consistency.

Actions taken: opened `https://eviltester.github.io/grid-table-editor/` in a Playwright CLI session, captured the landing-page snapshot, and recorded the visible branch/commit/build metadata plus top-level links into the app, generator, Storybook, and nested `site/`.

Observations: the deployed landing page is live on branch `codex/228-improve-command-definition`, commit `8382b9e1947b`, built `2026-06-24T16:07:45.755Z`; the top-level page exposes the nested docs path as `./site/`; the browser session also reported one console error on load, which may be unrelated to the docs charter but is worth checking if it recurs inside help or docs surfaces.

2026-06-24T17:50:30.0000000+01:00

- Intent: compare published generating-data docs and in-app help links against the live nested app and generator surfaces.

Actions taken: reviewed `/site/docs/intro`, expanded the `Generating Data` docs sidebar, opened `Method Picker UI Spec`, `Faker Based Data`, and `Domain Test Data`, then opened `/site/app.html` and expanded the `Test Data` help tooltip and schema panel.

Observations: nested docs routing is healthy and all checked doc links resolved under `/grid-table-editor/site/...`; the in-app `Test Data` help tooltip links to `/site/docs/test-data/test-data-generation`; regex and domain row help icons point to `/site/docs/test-data/regex-test-data` and `/site/docs/test-data/domain/domain-test-data`; the published docs now explicitly separate curated `domain.*` usage from faker-only `helpers.*`, including the migration warning that `domain.helpers.fake("...")` is invalid.

2026-06-24T17:52:10.1157676+01:00

- Intent: compare the published method-picker docs/spec with the live runtime controls and execute at least one documented example in the deployed generator.

Actions taken: inspected the standalone `/generator.html` schema controls, inspected the nested `/site/app.html` test-data schema controls, ran a live regex example in the standalone generator (`Code` + regex `[A-Z]{3}`) and previewed generated output, then switched the same row to `domain` to inspect the available live command-selection control.

Observations: the live runtime does not match the published `Method Picker UI Spec` page yet; both the standalone generator and nested app still use inline controls rather than the shared modal picker described by the spec page, and no visible modal-picker entry point was present during this pass. The standalone generator still shows a plain type dropdown for `enum/literal/regex/domain/faker`; the nested app improves on that for `domain` by exposing a searchable command dropdown plus params field, but this is still not the documented modal with tabs/search/details. The regex example executed successfully and previewed CSV output with header `Code` and sample values such as `TNW`, `GCJ`, and `XLT`, which confirms the deployed generator still works for representative documented examples even while the picker-spec docs appear ahead of the live UI.

---

ux-regression-test-log.md

UX Regression Test Log

2026-06-24T17:52:00.0000000+01:00

- Charter: assess workflow regression and usability across generator, method-picker, help, params editing, and related schema authoring flows in the deployed environment.

Techniques and heuristics to use: exploratory testing, state/flow modeling, friction hunting, consistency checks, and repeatability checks for workflow interruptions.

Expected focus: discoverability, clarity of examples and errors, help interactions, picker-to-row insertion flows, params editing, preview/generate loops, and any workflow regressions caused by the shared metadata path.

2026-06-24 17:49:22 +01:00

- intent: Prove live browser access on the deployed issue-228 / PR-243 environment and scout the available UX surfaces before filing any regressions.
- actions: Opened `https://eviltester.github.io/grid-table-editor/` in a real browser session via Playwright CLI; confirmed the published branch/build metadata; followed the live `Open generator.html` entry point; captured the initial generator surface inventory including help affordances, schema editing controls, output settings, preview, and data-table preview.
- observations: Browser control works against the deployed environment. The landing page shows branch `codex/228-improve-command-definition`, commit `8382b9e1947b`, built `2026-06-24T16:07:45.755Z`. The generator surface loads with visible help triggers, schema row editing, output-format controls, preview controls, and nested docs/help links. No regression claimed from the scout pass.

2026-06-24 17:51:42 +01:00

- intent: Exercise the schema method picker and params editing flow for a non-trivial faker command, then verify whether the resulting rule can be previewed through the standard authoring loop.
- actions: Opened the top-level generator help and confirmed the contextual overview tooltip rendered; switched the first schema row from `regex` to `faker`; opened the method picker; selected `helpers.rangeToNumber`; filled the row column name as `count`; attempted a direct preview from the row-level params textbox; then opened the structured params editor and entered `{ min: 1, max: 9 }` for the required `numberOrRange` value.

- observations: The method picker itself is usable and exposes descriptions, parameter metadata, examples, and documentation links. The follow-on params workflow is broken for this object-typed faker helper: the inline params textbox did not visibly retain a typed object value before preview, the preview still generated large unrelated numeric-looking values, and the structured params editor produced `Generated params (numberOrRange={ min: 1, max: 9 })` but blocked Apply with `Row 1: invalid faker params - Invalid Faker API Call Unsafe faker rule syntax detected: requires complex argument parsing`. This makes a picker-exposed command effectively unconfigurable through the UI. Screenshot saved: `screenshots/ux-regression-faker-param-editor-invalid-object.png`.

2026-06-24 17:52:22 +01:00

- intent: Sanity-check the adjacent help surface for the selected faker command so the params failure can be compared against the guidance shown in-product.
- actions: Re-opened the generator state after cancelling the params dialog; used the selected command help affordance for `helpers.rangeToNumber`; inspected the rendered command tooltip content and its embedded example syntax.
- observations: The in-product help compounds the params-editor failure rather than explaining it. The tooltip explicitly presents the object form as valid example syntax (`helpers.rangeToNumber({ min: 1, max: 2 })`) and describes the param type as `number | { min: number; max: number; }`, which directly conflicts with the structured params editor refusing the same object shape as "unsafe faker rule syntax". This is a usability contradiction across method-picker help and params editing, not just a single validation message.

---

responsive-accessibility-test-log.md

Responsive Accessibility Test Log

2026-06-24T17:52:00.0000000+01:00

- Charter: review the deployed generator/app/help surfaces under responsive and accessibility heuristics, including keyboarding and narrow-width behavior.

Techniques and heuristics to use: responsive testing heuristics, accessibility heuristics, keyboard-only exploration, focus-order checks, and visible-state consistency checks.

Expected focus: mobile and tablet layouts, keyboard reachability, focus visibility, modal/help interactions, semantics exposed through the accessibility tree, and user-visible layout breakage.

2026-06-24T18:01:00+01:00

- Intent: prove live browser access against the deployed environment and scout which surfaces best match the responsive/accessibility charter.

Actions: opened `https://eviltester.github.io/grid-table-editor/` and `https://eviltester.github.io/grid-table-editor/app.html` in a standalone Playwright browser session against the live GitHub Pages deployment; captured the visible top-level links/buttons/inputs inventory from the rendered page state to identify responsive, keyboard, and help-heavy surfaces worth deeper coverage.

Observations: browser control worked against the deployed site. The root landing page is a lightweight launcher, while `app.html` exposes the highest-density interactive surface for this charter: icon-only toolbar buttons, multiple help triggers, filter/sort controls, schema/import-export sections, preview/copy actions, and modal-capable controls. Several actionable controls rely on `aria-label` rather than visible text, which makes keyboard/focus verification especially important in the next passes.

2026-06-24T18:12:00+01:00

- Intent: check whether the deployed app remains usable at mobile/tablet/narrow widths and capture visible layout/accessibility evidence.

Actions: opened `app.html` at `375x812`, `320x800`, and `768x1024`, plus `/site/` at `375x812`; recorded viewport-vs-scroll-width metrics; saved screenshots `responsive-accessibility-mobile-app-375.png`, `responsive-accessibility-narrow-app-320.png`, `responsive-accessibility-tablet-app-768.png`, and `responsive-accessibility-mobile-site-375.png`; visually reviewed the captured images.

Observations: the app avoided horizontal page overflow at the tested widths and stayed functionally visible on mobile/tablet, but the mobile layout becomes extremely dense. Many primary controls in the live app render at roughly `21-22px` tall, and several icon-only controls/help icons shrink to roughly `13-22px`, which is well below comfortable mobile tap sizing. The docs/home page also

remained readable on mobile, although the hero heading wraps aggressively into a stacked multi-line block rather than preserving a more balanced small-screen headline.

2026-06-24T18:19:00+01:00

- Intent: verify keyboard reachability, focus visibility/order, and help/accordion behavior on the mobile viewport.

Actions: ran a keyboard-only `Tab` pass through the live `app.html` mobile view, logging the first 45 focus stops; captured focused-state screenshots including `responsive-accessibility-mobile-tab-2.png`, `responsive-accessibility-mobile-tab-10.png`, `responsive-accessibility-mobile-tab-23.png`, `responsive-accessibility-mobile-tab-24.png`, and `responsive-accessibility-mobile-tab-28.png`; specifically rechecked collapsed `Test Data` and `Import / Export` sections to see whether they leaked hidden focusable descendants.

Observations: focus indicators were generally visible on the tested controls, so the page does show keyboard focus styling. The strongest defect found is that collapsed sections still expose nested help controls in the tab order: when `Test Data` is still collapsed, the next `Tab` after its summary lands on the section help icon (`responsive-accessibility-mobile-tab-24.png`), and when `Import / Export` is collapsed, `Tab` later jumps into a hidden option-help control and scrolls the page deep into that closed section (`responsive-accessibility-mobile-tab-28.png`). That creates confusing keyboard order and exposes tooltip content from sections the user has not opened. The screenshots also reinforce the target-size issue: keyboard-focusable help icons and header action buttons are visibly tiny on mobile, even though they do receive focus.

---

enum-cross-surface-test-log.md

Enum Cross-Surface Test Log

2026-06-24T17:52:00.0000000+01:00

- Charter: trace enum behavior across row mode, text mode, domain-command usage, preview/generation, and export-oriented surfaces in the deployed branch.

Techniques and heuristics to use: state/flow modeling, positive and negative sampling, normalization/oracle checking, and repeatability checks across multiple

enum spellings and syntactic shapes.

Expected focus: `enum(...)`, `enum value1,value2`, `datatype.enum(...)`, `awd.datatype.enum(...)`, displayed versus saved syntax, cross-surface parity between app and generator, and any export/rendering regressions tied to enum normalization.

2026-06-24T17:48:06.2114860+01:00

- Intent: prepare the deployed-only enum cross-surface session, capture prior expectations, and confirm the browser route before interacting with the live app.

Actions:

- Read the existing charter block in this owned log to preserve the requested scope.
- Checked prior memory for issue #228 enum/domain cleanup and prior deployed exploratory browser work.
- Confirmed Playwright skill instructions and verified that `npx` is available for live browser automation.
- Checked the screenshots folder so new captures can follow the required `enum-cross-surface-` naming pattern.

Observations:

- Prior implementation notes indicate internal normalization should favor `datatype.enum(...)` while public schema text should round-trip as `enum(...)`.
- Prior deployed-browser notes favor Playwright over Chrome DevTools for reliable interaction with `https://eviltester.github.io/grid-table-editor/`.
- The screenshots folder already contains earlier browser-proof images; new captures should use the `enum-cross-surface-` prefix to stay distinct.

2026-06-24T17:52:12.3792894+01:00

- Intent: prove live access to the deployed issue #228 / PR #243 environment, confirm the exact published build, and inventory the app and generator surfaces most relevant to enum behavior.

Actions:

- Opened `https://eviltester.github.io/grid-table-editor/` with Playwright CLI using a dedicated `enum228` session after browser MCP attach paths proved unreliable.

- Captured the live landing-page snapshot and verified the published branch, commit, and build timestamp.
- Opened `app.html` and `generator.html` in separate tabs and captured accessibility snapshots of each surface.
- Inspected the app export/preview region and the generator schema editor, preview region, and output-format controls for enum-relevant entry points.

Observations:

- Live browser access is proven against the deployed site; the test-environment landing page reported branch `codex/228-improve-command-definition`, commit `8382b9e1947b`, built `2026-06-24T16:07:45.755Z`.
- The generator exposes direct schema row editing plus a text-mode toggle, which made it the fastest surface for live enum syntax checks.
- The app surface clearly exposes Test Data options plus export-oriented preview links (Markdown, CSV, Delimited, JSON, JSONL, XML, SQL, Code, Gherkin, HTML, ASCII), but I did not complete a full app-side enum authoring flow before wrap-up.
- Chrome/DevTools-style MCP attach remained flaky because of an already-running browser profile; Playwright CLI was the dependable route for this deployed-only pass.

2026-06-24T17:52:12.3792894+01:00

- Intent: trace enum behavior in the generator across row mode and text mode, establish a working baseline, and sample shorthand syntax acceptance.

Actions:

- In generator row mode, filled `Column Name = color`, changed the rule type from `regex` to `enum`, and entered `red,green,blue` in the value field.
- Ran `Preview` and inspected both the text preview and the data-table preview.
- Switched the same schema from row mode into text mode with `Edit as Text` and recorded the rendered schema text.
- Captured a screenshot of the working generator text-mode baseline and copied it into the owned screenshots folder as `enum-cross-surface-generator-text-baseline.png`.
- Replaced the text-mode schema with the shorthand variant `color enum red,green,blue` and ran `Preview` again to see whether the no-parentheses form would parse.

Observations:

- Row mode with `enum` plus comma-separated values worked in the deployed generator: preview rows were limited to the supplied set (`red`, `green`, `blue`) and the output preview populated successfully.

- When the same working row-mode schema was opened in text mode, the public text representation rendered as `color enum(red,green,blue)`.
- This is consistent with the expected public-facing normalization toward `enum(...)` rather than exposing an internal `datatype.enum(...)` form in text mode.
- The shorthand text variant `color enum red,green,blue` did not parse in this surface. The live error was `column color enum red,green,blue requires a data definition, use 'literal("")' for blank data`, and preview output was cleared.
- Because of time, I did not finish practical live checks for `datatype.enum(...)` or `awd.datatype.enum(...)`, nor did I complete export-format switching or app-side enum authoring before wrap-up.

---

defects\defect-001-datatype-enum-missing-from-published-docs.md

Defect 001: `datatype.enum` Missing From Published Docs Surfaces

Summary

The deployed runtime and generator UI expose `datatype.enum`, but the published datatype domain docs and the published method-picker UI spec docs do not mention `datatype.enum`.

Environment

- Deployed test environment: `https://eviltester.github.io/grid-table-editor/`
- Session date: `2026-06-24`
- Branch shown on landing page: `codex/228-improve-command-definition`
- Commit shown on landing page: `8382b9e1947b`

Repeatability

Repeatable.

Steps To Reproduce

1. Open `https://eviltester.github.io/grid-table-editor/generator.html`.
2. Change the first schema row to `domain`.
3. Open or inspect the domain command picker list.
4. Observe that `datatype.enum` is present in the command list.
5. Open `https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/datatype`
6. Inspect the published page content or fetch the page HTML.

7. Open `https://eviltester.github.io/grid-table-editor/site/docs/test-data/method-picker-u`
8. Inspect the published page content or fetch the page HTML.

Expected Result

Published docs/help surfaces should document `datatype.enum` when the deployed runtime and UI expose it as a supported command.

Actual Result

- The generator domain picker exposes `datatype.enum`.
- The published datatype domain docs do not contain `datatype.enum`.
- The published method-picker UI spec docs do not contain `datatype.enum`.

Evidence

- Runtime/UI exposure screenshot: main-loop1-domain-picker.png
- Deployed datatype docs screenshot: main-loop1-datatype-doc.png
- HTML verification captured during the session:
  - datatype docs HTML contained `datatype.boolean` but not `datatype.enum`
  - method-picker UI spec HTML did not mention `datatype.enum`

Why This Matters

This creates a docs/help drift defect on the same deployed branch that introduces and normalizes `datatype.enum`. Users can discover and execute the command in the runtime, but cannot find matching published guidance in the most obvious documentation surfaces.

defects\defect-002-malformed-explicit-enum-silently-treated-as-literal.md

Defect 002: Malformed Explicit Enum Syntax Is Silently Treated As Literal Output

Summary

When the schema text contains malformed explicit enum syntax such as `datatype.enum(unclosed`, the deployed generator does not surface a validation error. Instead, preview generation succeeds and emits the malformed text literally as generated data.

Environment

- Deployed test environment: `https://eviltester.github.io/grid-table-editor/generator.html`

- Session date: `2026-06-24`
- Output format used during confirmation: `MARKDOWN`

Repeatability

Repeatable.

Steps To Reproduce

1. Open `https://eviltester.github.io/grid-table-editor/generator.html`.
2. Switch the schema editor to text mode.
3. Set the output format to `MARKDOWN` or leave it on any previewable format.
4. Enter this schema text:

```
Method
datatype.enum(unclosed
```

5. Click `Preview`.

Expected Result

The generator should reject malformed explicit enum syntax and show a validation or parsing error instead of generating data.

Actual Result

Preview succeeds and outputs the literal string `datatype.enum(unclosed` repeatedly as generated values for the `Method` column.

Evidence

- Row-to-text round-trip screenshot for the valid baseline: main-loop3-row-to-text-roundtrip.png
- The malformed case was captured in the generator preview state during Loop 3, where the output preview showed repeated literal values:
  - `datatype.enum(unclosed`
  - `datatype.enum(unclosed`
  - `datatype.enum(unclosed`

Why This Matters

PR `#243` explicitly refactors enum handling through shared parsing and normalization. Accepting malformed explicit enum syntax as literal text hides a real authoring mistake, defeats validator expectations, and can silently leak invalid schema intent into generated outputs.

defects\defect-003-faker-range-to-number-object-params-blocked.md

Defect 003: `helpers.rangeToNumber` Object Params Are Exposed In UI Help But Blocked By Params Validation

Summary

The deployed generator exposes `helpers.rangeToNumber` through the faker method picker and documents an object-shaped argument, but the structured params editor rejects that documented object form and disables `Apply`.

Environment

- Deployed test environment: `https://eviltester.github.io/grid-table-editor/generator.html`
- Session date: `2026-06-24`

Repeatability

Repeatable.

Steps To Reproduce

1. Open `https://eviltester.github.io/grid-table-editor/generator.html`.
2. Change the first schema row type to `faker`.
3. Open the method picker and select `helpers.rangeToNumber`.
4. Open the structured params editor.
5. Enter the documented object-shaped value `{ min: 1, max: 9 }` for `numberOrRange`.
6. Try to apply the params.
7. Open the in-product help for the selected command and compare the documented syntax.

Expected Result

If the picker/help surface documents `helpers.rangeToNumber({ min: 1, max: 2 })`, the params editor should accept the same object shape or the help should clearly document a different supported syntax.

Actual Result

- The structured params editor shows generated params based on the object shape.
- `Apply` remains disabled.

- The row reports `Unsafe faker rule syntax detected: requires complex argument parsing`.
- The in-product help still teaches the object-shaped syntax and describes the param type as `number | { min: number; max: number; }`.

Evidence

- Screenshot: ux-regression-faker-param-editor-invalid-object.png
- Supporting log: ux-regression-test-log.md

Why This Matters

This is a direct workflow contradiction inside the deployed UI: a method is discoverable and documented in the picker/help flow, but the documented configuration path is blocked by the same UI's validation rules.

defects\defect-004-collapsed-sections-leak-hidden-focus-targets.md

Defect 004: Collapsed Sections Leak Hidden Focus Targets Into Keyboard Order On Mobile

Summary

On mobile `app.html`, collapsed sections still expose nested focusable controls to keyboard users. Tabbing moves into hidden help targets inside collapsed `Test Data` and `Import / Export` sections, causing confusing focus order and unexpected tooltip content from content the user has not opened.

Environment

- Deployed test environment: `https://eviltester.github.io/grid-table-editor/app.html`
- Session date: `2026-06-24`
- Mobile viewport coverage used during confirmation: `375x812` and `320x800`

Repeatability

Repeatable.

Steps To Reproduce

1. Open `https://eviltester.github.io/grid-table-editor/app.html` at a mobile-width viewport.
2. Leave the `Test Data` section collapsed.
3. Use keyboard `Tab` navigation forward through the page.
4. Continue tabbing until focus reaches the collapsed `Test Data` summary.

5. Tab again and observe focus moving into a hidden nested help target.
6. Continue the same process with `Import / Export` still collapsed.

Expected Result

Collapsed sections should not expose hidden descendant controls in the normal keyboard tab order.

Actual Result

- After the collapsed `Test Data` summary, `Tab` lands on a hidden nested help icon.
- Later, while `Import / Export` is still collapsed, `Tab` reaches a hidden option-help target and scrolls the page into the closed region.
- Tooltip content can appear from closed sections that the user has not opened.

Evidence

- responsive-accessibility-mobile-tab-24.png
- responsive-accessibility-mobile-tab-28.png
- Supporting log: responsive-accessibility-test-log.md

Why This Matters

This is a real keyboard accessibility defect. It breaks expected focus order, makes navigation confusing on mobile, and exposes controls that are visually and structurally hidden.